

Раздел 1. Цель работы

Цель работы - реализовать загрузку очищенных данных в спроектированное хранилище и выполнить миграцию данных из плоских файлов в реляционную базу данных.

Задачи работы:

1. установить и настроить СУБД PostgreSQL;
2. создать базу данных и таблицы по спроектированной схеме;
3. загрузить данные из CSV-файлов в таблицы;
4. проверить целостность и полноту загруженных данных;
5. создать индексы для оптимизации запросов.

Загружается очищенный набор `data_cleaned.csv` (10 000 записей, 28 полей), полученный ETL-процессом в практической работе № 4, в хранилище со схемой «звезда», спроектированное в практической работе № 5: пять измерений и две таблицы фактов.

Раздел 2. Настройка инфраструктуры

Использована свободная СУБД PostgreSQL - рекомендованный методическими указаниями вариант. Установка выполнена из пакетов дистрибутива Linux (вариант А). Для воспроизведения на другом компьютере подготовлен файл `docker-compose.yml` (вариант Б): команда `docker compose up -d` поднимает тот же сервер с теми же параметрами.

Сервер настроен так, чтобы подключение было как к реальному серверу: доступ по TCP (`localhost:5432`) только с паролем (метод `scram-sha-256` в `pg_hba.conf`). Параметры подключения приведены в таблице 1, версии ПО - в таблице 2.

Таблица 1 - Параметры подключения к базе данных

Параметр	Значение	Переменная окружения
Хост	localhost	DB_HOST
Порт	5432	DB_PORT
База данных	ai_project	DB_NAME
Пользователь	postgres	DB_USER

Параметр	Значение	Переменная окружения
Пароль	задаётся при установке	DB_PASSWORD
Схема хранилища	dwh	-
Аутентификация	scram-sha-256 (по паролю)	-

Пароль не записывается в код: скрипты читают параметры из переменных окружения, а значения по умолчанию совпадают с примером из методических указаний.

Таблица 2 - Версии программного обеспечения

Компонент	Версия	Назначение	Лицензия
PostgreSQL	16.14	СУБД хранилища	PostgreSQL License
psql	16.14	Консольный клиент: проверочные запросы	PostgreSQL License
Python	3.11.15	Скрипты загрузки и проверки	PSF License
pandas	3.0.6	Чтение CSV, построение таблиц измерений и фактов	BSD-3-Clause
SQLAlchemy	2.1.3	Подключение, транзакции, to_sql	MIT
psycopg2	2.9.13	Драйвер PostgreSQL, команда COPY	LGPL
Docker (необязательно)	postgres:16	Альтернативный способ развертывания	Apache-2.0

Создание базы данных и таблиц:

```
CREATE DATABASE ai_project;
\c ai_project
\i ../лаба_5/sql/01_create_schema.sql -- схема dwh, 7 таблиц
\i ../лаба_5/sql/02_reference_data.sql -- справочные измерения
\i ../лаба_5/sql/03_views.sql -- аналитические представления
```

В программе загрузки эти же скрипты выполняются автоматически в начале транзакции. Результат проверки командой \dt+ показан на рисунке 1: созданы семь таблиц с комментариями из DDL.

Schema	Name	Type	Owner	Persistence	Access method	Size	Description
dwh	dim_batch	table	postgres	permanent	heap	8192 bytes	Измерение: партия загрузки (прослеживаемость)
dwh	dim_equipment	table	postgres	permanent	heap	8192 bytes	Измерение: единица оборудования (фрейм-экземпляр, пороги правил)
dwh	dim_failure_type	table	postgres	permanent	heap	8192 bytes	Измерение: тип отказа и соответствующее производственное правило
dwh	dim_quality_class	table	postgres	permanent	heap	8192 bytes	Измерение: класс качества изделия (фрейм-прототип)
dwh	dim_tool	table	postgres	permanent	heap	8192 bytes	Измерение: цикл эксплуатации режущего инструмента
dwh	fact_failure_event	table	postgres	permanent	heap	24 kB	Факт: событие отказа (подтип отказа в наблюдении)
dwh	fact_observation	table	postgres	permanent	heap	1496 kB	Факт: наблюдение за станком при обработке одного изделия

(7 rows)

Рисунок 1 - Таблицы хранилища (psql, \dt+)

Раздел 3. Процесс загрузки данных

3.1. Выбор способа загрузки

Перед написанием программы загрузки четыре способа загрузки из методических указаний сравнены на одних и тех же 10 000 строках таблицы фактов (программа `bench_load.py`, медиана трёх запусков, таблица 3).

Таблица 3 - Сравнение способов загрузки 10 000 строк

Способ	Время, с	Относительно COPY
INSERT по одной строке	1,519	в 11,7 раза медленнее
<code>pandas.to_sql</code> (<code>executemany</code>)	1,010	в 7,8 раза медленнее
INSERT множественный (<code>to_sql</code> , <code>method=multi</code>)	4,906	в 37,8 раза медленнее
COPY FROM STDIN	0,130	-

Команда COPY оказалась самой быстрой - 0,13 с, в 12 раз быстрее построчной вставки: СУБД получает данные одним потоком и не разбирает отдельную команду на каждую строку. Неожиданный результат дал множественный INSERT: в SQLAlchemy 2 обычный `to_sql` уже группирует строки (режим `insertmanyvalues`), а параметр `method=multi` строит одну огромную команду с десятками тысяч параметров, разбор которой обходится дороже. Поэтому выбрана комбинация: таблицы фактов загружаются командой COPY, небольшие измерения - через `pandas.to_sql`, что короче в коде и на 120 строках по времени не отличается.

3.2. Скрипт загрузки

Программа `load_dwh.py` выполняет загрузку в одной транзакции. Если любой шаг завершится ошибкой (нарушение ограничения, обрыв соединения), транзакция откатывается и хранилище остаётся в исходном состоянии - частично загруженных данных не бывает. Основные фрагменты:

```
DB_NAME = os.getenv('DB_NAME', 'ai_project')          # параметры - из окружения
DB_PASSWORD = os.getenv('DB_PASSWORD', 'mysecretpassword')
engine = create_engine(f'postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}')
df = pd.read_csv('../Лаба_4/data/data_cleaned.csv')
md5 = hashlib.md5(SRC.read_bytes()).hexdigest()

with engine.begin() as conn:                          # одна транзакция на всю загрузку
    for f in ['01_create_schema.sql', '02_reference_data.sql', '03_views.sql']:
        run_sql_file(conn, LAB5 / f)
```

```

# партия загрузки: ключ возвращает СУБД
batch_key = conn.execute(text(
    'INSERT INTO dwh.dim_batch (source_name, source_file, source_md5,
etl_script) '
    'VALUES (:n, :f, :m, :s) RETURNING batch_key'), {...}).scalar_one()
# измерение циклов инструмента - агрегация по tool_cycle_id
dim_tool = build_dim_tool(df)
dim_tool.to_sql('dim_tool', conn, schema='dwh', if_exists='append',
index=False)
# факты: бизнес-значения заменяются суррогатными ключами измерений
fact = build_fact_observation(df, batch_key)
copy_df(conn, fact, 'fact_observation')
events = build_fact_failure_event(df, batch_key)
copy_df(conn, events, 'fact_failure_event')
conn.exec_driver_sql('ANALYZE') # статистика для планировщика

```

Загрузка командой COPY через драйвер psycopg2:

```

def copy_df(conn, df, table):
    buf = io.StringIO()
    df.to_csv(buf, index=False, header=False)
    buf.seek(0)
    cols = ','.join(df.columns)
    with conn.connection.cursor() as cur:
        cur.copy_expert(f'COPY dwh.{table} ({cols}) FROM STDIN WITH (FORMAT csv)',
buf)

```

Построение измерения циклов инструмента и таблицы событий отказа:

```

def build_dim_tool(df):
    g = df.groupby('tool_cycle_id')
    tool = pd.DataFrame({'tool_key': g.size().index, 'tool_cycle_no':
g.size().index,
                        'first_udi': g['udi'].min().values,
                        'last_udi': g['udi'].max().values,
                        'products_processed': g.size().values,
                        'wear_at_end_min': g['tool_wear_min'].last().values,
                        'end_twf': g['twf'].last().values})
    tool['end_reason'] = 'PLANNED'
    tool.loc[tool['end_twf'] == 1, 'end_reason'] = 'TWf'
    tool.loc[tool.index[-1], 'end_reason'] = 'IN_SERVICE' # цикл не завершён
    return tool.drop(columns='end_twf')

def build_fact_failure_event(df, batch_key): # строка на каждый подтип отказа
    for col, code in [('twf', 'TWf'), ('hdf', 'HDF'), ('pwf', 'PWF'), ('osf',
'OSF'),
                      ('rnf_ref', 'RNF'), ('failure_type_unknown', 'UNKNOWN')]:
        sub = df[df[col] == 1]
        ... # udi, ключи измерений, is_primary = (failure_mode == code)

```

Полный текст программы приведён в приложении А.

3.3. Результаты загрузки

Загрузка всего хранилища, включая создание схемы, выполняется за 0,83 с. Количество загруженных записей приведено в таблице 4; ожидаемые значения рассчитаны по исходному CSV-файлу.

Таблица 4 - Количество загруженных записей по таблицам

Таблица	Тип	Загружено	Ожидалос ь	Способ
dim_equipment	Измерение	1	1	SQL-скрипт

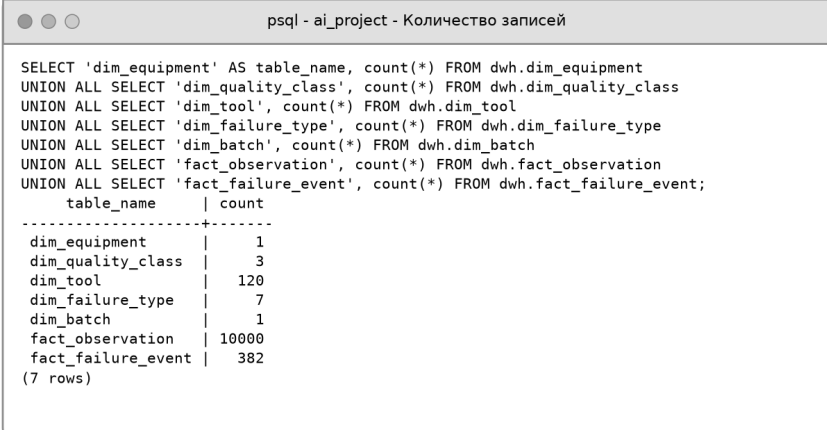
Таблица	Тип	Загружено	Ожидалось	Способ
dim_quality_class	Измерение	3	3	SQL-скрипт
dim_failure_type	Измерение	7	7	SQL-скрипт
dim_tool	Измерение	120	120	to_sql
dim_batch	Измерение	1	1	INSERT ... RETURNING
fact_observation	Факт	10 000	10 000	COPY
fact_failure_event	Факт	382	382	COPY

Все таблицы загружены полностью. При вставке СУБД проверила все ограничения из лабы 5 - внешние ключи, допустимые диапазоны показаний, бинарность меток и согласованность обобщённой метки с подтипами: ни одна из 10 000 строк их не нарушила.

Раздел 4. Проверка целостности

4.1. Полнота и ссылочная целостность

Проверочные запросы собраны в файл sql/05_checks.sql и выполняются как в psql, так и программой check_integrity.py. Количество записей по таблицам показано на рисунке 2.



```

psql - ai_project - Количество записей

SELECT 'dim_equipment' AS table_name, count(*) FROM dwh.dim_equipment
UNION ALL SELECT 'dim_quality_class', count(*) FROM dwh.dim_quality_class
UNION ALL SELECT 'dim_tool', count(*) FROM dwh.dim_tool
UNION ALL SELECT 'dim_failure_type', count(*) FROM dwh.dim_failure_type
UNION ALL SELECT 'dim_batch', count(*) FROM dwh.dim_batch
UNION ALL SELECT 'fact_observation', count(*) FROM dwh.fact_observation
UNION ALL SELECT 'fact_failure_event', count(*) FROM dwh.fact_failure_event;
 table_name | count
-----+-----
 dim_equipment |      1
 dim_quality_class |      3
 dim_tool |    120
 dim_failure_type |      7
 dim_batch |      1
 fact_observation | 10000
 fact_failure_event |    382
(7 rows)

```

Рисунок 2 - Количество записей по таблицам

Проверка висячих ссылок выполнена для каждого внешнего ключа: левое соединение таблицы фактов с измерением и подсчёт строк, для которых строка измерения не найдена (таблица 5). Пропуски в ключевых полях проверены отдельно (рисунок 3).

Таблица 5 - Результаты проверки ссылочной целостности

Проверяемая связь	Строк без пары
equipment	0
quality_class	0
tool	0
failure_type	0
batch	0
event -> observation	0
NULL в ключевых полях fact_observation	0

```

psql - ai_project - Висячие ссылки и NULL в ключах

SELECT count(*) AS orphan_class FROM dwh.fact_observation f
LEFT JOIN dwh.dim_quality_class d ON f.class_key = d.class_key WHERE d.class_key IS NULL;
orphan_class
-----
0
(1 row)

SELECT count(*) AS orphan_tool FROM dwh.fact_observation f
LEFT JOIN dwh.dim_tool d ON f.tool_key = d.tool_key WHERE d.tool_key IS NULL;
orphan_tool
-----
0
(1 row)

SELECT count(*) AS null_keys FROM dwh.fact_observation
WHERE udi IS NULL OR class_key IS NULL OR tool_key IS NULL OR failure_type_key IS NULL;
null_keys
-----
0
(1 row)

```

Рисунок 3 - Проверка висячих ссылок и пропусков в ключах

4.2. Сверка с источником

Базовая статистика показаний, рассчитанная в СУБД (рисунок 4), совпадает со статистикой из лабы 2 (таблица 3): среднее значение температуры воздуха - 300,00 К, частоты вращения - 1538,78 об/мин, крутящего момента - 39,99 Н·м, износа - 107,95 мин; минимум и максимум момента - 3,8 и 76,6 Н·м.

```

psql - ai_project - Базовая статистика

SELECT min(torque_nm) AS min_torque, max(torque_nm) AS max_torque,
round(avg(torque_nm), 2) AS avg_torque, round(stddev(torque_nm), 2) AS stddev_torque,
sum(machine_failure) AS failures
FROM dwh.fact_observation;
min_torque | max_torque | avg_torque | stddev_torque | failures
-----+-----+-----+-----+-----
3.8 | 76.6 | 39.99 | 9.97 | 339
(1 row)

SELECT * FROM dwh.v_failure_rate_by_class;
class_code | observations | failures | failure_rate_pct | twf | hdf | pwf | osf
-----+-----+-----+-----+-----+-----+-----+-----
L | 6000 | 235 | 3.92 | 25 | 76 | 59 | 87
M | 2997 | 83 | 2.77 | 14 | 31 | 31 | 9
H | 1003 | 21 | 2.09 | 7 | 8 | 5 | 2
(3 rows)

```

Рисунок 4 - Базовая статистика и доля отказов по классам качества

Помимо агрегатов, выполнена поэлементная сверка: программа выгружает представление `v_observation` и сравнивает каждое значение с CSV-файлом. Совпадение - во всех 10 000 записях по всем проверенным полям (показания, мощность, произведение износа и момента, метка отказа, основной подтип). Доли отказов по классам (3,92 / 2,77 / 2,09 %) совпадают с таблицей 6 лабы 2.

Согласованность двух таблиц фактов: сумма флагов подтипов в `fact_observation` (382) равна числу строк `fact_failure_event` (382), а число основных событий (339) - числу отказов (339).

4.3. Проверка ограничений СУБД

Чтобы убедиться, что ограничения целостности действительно работают, выполнены шесть заведомо некорректных вставок, каждая в своей транзакции (таблица 6).

Таблица 6 - Результаты негативных тестов

Попытка	Реакция СУБД
Несуществующий класс качества (<code>class_key = 99</code>)	ForeignKeyViolation
Температура 27 К вместо 300 К (ошибка единиц)	CheckViolation
Отказ без подтипа и без флага <code>failure_type_unknown</code>	CheckViolation
Повтор UDI существующего наблюдения	UniqueViolation
Повторная загрузка того же файла (тот же MD5)	UniqueViolation
Событие отказа для несуществующего наблюдения	ForeignKeyViolation

Все шесть попыток отклонены. Число строк в `fact_observation` до тестов - 10 000, после - 10 000: транзакции с ошибкой откатились полностью. Ограничение UNIQUE на MD5 исходного файла делает загрузку идемпотентной: случайный повторный запуск не удвоит данные.

4.4. Проверка производционных правил в хранилище

Представление `v_rule_check` вычисляет правила по порогам, хранящимся в измерениях. Результат сопоставлен с таблицей 7 лабы 2 (таблица 7).

Таблица 7 - Правила в хранилище и в лабе 2

Правило	Сработало (БД / лаба 2)	Совпадений (БД / лаба 2)	Полнота, %	Точность, %
HDF	103 / 115	103 / 115	89,6	100,0
PWF	95 / 95	95 / 95	100,0	100,0
OSF	98 / 98	98 / 98	100,0	100,0
TWF	790 / 790	43 / 43	93,5	5,4

Правила PWF, OSF и TWF дают в точности те же результаты, что и в лабе 2. Правило HDF сработало на 103 записях вместо 115 - это главная находка этапа. Причину установила программа `check_integrity.py`: 27 записей имеют разность температур ровно 8,6 К при оборотах ниже 1380 об/мин, то есть лежат точно на границе правила. Метка HDF стоит у 12 из них, и это ровно те 12 записей, у которых разность, вычисленная в двоичной арифметике с плавающей точкой, равна 8,599999... вместо 8,6. Генератор набора AI4I сравнивал числа с плавающей точкой, и метки на границе определены ошибкой округления. В лабе 2 pandas вычислял разность так же, во float, поэтому повторил эту ошибку и показал 100 %. PostgreSQL хранит температуры в типе NUMERIC и вычисляет разность точно.

Ни строгое ($< 8,6$), ни нестрогое ($\leq 8,6$) условие на точных данных метки не воспроизводит: первое даёт полноту 89,6 % при точности 100 %, второе - полноту 100 % при точности 88,5 %. Практический вывод для базы знаний: 27 граничных записей - зона неопределённости шириной меньше точности датчика (0,1 К). Правило HDF следует применять в нестрогой форме ($\leq 8,6$ К): пропуск отказа теплоотвода опаснее ложной тревоги. Граничным случаям нужно назначать пониженный коэффициент уверенности, а не считать их однозначными. Эффект проявился именно при переносе данных в СУБД с точной десятичной арифметикой.

Раздел 5. Оптимизация

5.1. Созданные индексы

Уникальные индексы на `udi`, `product_id`, бизнес-ключи измерений и пару (`udi`, `failure_type_key`) создала сама СУБД по ограничениям из лабы 5.

PostgreSQL не индексирует внешние ключи автоматически, поэтому дополнительные индексы созданы по типовым запросам системы (таблица 8, файл sql/04_indexes.sql).

Таблица 8 - Созданные индексы

Индекс	Столбцы	Размер	Назначение
idx_event_tool	(tool_key)	16 kB	События по циклу инструмента
idx_event_type	(failure_type_key)	16 kB	События по типу отказа
idx_obs_class_failure	(class_key, machine_failure)	88 kB	Фильтр «класс качества + отказ» на панели мониторинга
idx_obs_failure_type	(failure_type_key)	88 kB	Выборка наблюдений по типу отказа
idx_obs_failures	(udi) ГДЕ (machine_failure = 1)	16 kB	Частичный: только 339 записей с отказом - журнал отказов
idx_obs_tool	(tool_key)	88 kB	История цикла инструмента в карточке оборудования (СИ-2)
idx_obs_wear	(tool_wear_min)	88 kB	Диапазон износа - кандидаты на замену инструмента

Индексы увеличили объём таблицы fact_observation вместе с индексами с 2624 КБ до 2992 КБ. Частичный индекс idx_obs_failures занимает всего 16 КБ, поскольку содержит только 3,4 % строк.

5.2. Результаты EXPLAIN ANALYZE

Для пяти типовых запросов план и время выполнения получены командой EXPLAIN ANALYZE до и после создания индексов (программа optimize.py, медиана семи запусков). Таблица из 10 000 строк занимает около 1,8 МБ и целиком находится в памяти, поэтому, чтобы оценить эффект при объёме из нефункционального требования № 6 лабы 1 (2 млн записей), измерения повторены на нагрузочной копии таблицы: те же строки, размноженные 200 раз во временной схеме bench, которая после замеров удаляется (таблица 9).

Таблица 9 - Время выполнения запросов до и после создания индексов, мс

Запрос	10 тыс.: до	10 тыс.: после	2 млн: до	2 млн: после	План после
Q1. История цикла инструмента (карточка, СИ-2)	0,671	0,028	43,7	0,02	Index Scan
Q2. Журнал отказов	1,137	0,108	56,5	28,61	Index Scan
Q3. Отказы класса Н (панель мониторинга)	1,059	0,066	46,9	18,34	Index Scan

Запрос	10 тыс.: до	10 тыс.: после	2 млн: до	2 млн: после	План после
Q4. Кандидаты на замену инструмента: износ 230-240 мин	0,778	0,016	53,2	2,34	Bitmap Index Scan
Q5. Доля отказов по классам (агрегация всей таблицы)	2,280	2,201	155,8	145,86	Seq Scan

Тексты запросов:

Q1: SELECT * FROM fact_observation WHERE tool_key = 57;
Q2: SELECT udi, failure_type_key FROM fact_observation WHERE machine_failure = 1;
Q3: SELECT count(*) FROM fact_observation WHERE class_key = 3 AND machine_failure = 1;
Q4: SELECT udi, tool_wear_min FROM fact_observation WHERE tool_wear_min BETWEEN 230 AND 240;
Q5: SELECT class_key, avg(machine_failure) FROM fact_observation GROUP BY class_key;

План запроса Q1 до и после создания индекса показан на рисунке 5. До создания индекса СУБД читает всю таблицу (Seq Scan) и отбрасывает фильтром 9 919 строк из 10 000; после - находит 81 строку цикла инструмента по индексу (Index Scan).

```

psql - ai_project - EXPLAIN ANALYZE до и после индекса

BEGIN;
BEGIN
DROP INDEX dwh.idx_obs_tool;
DROP INDEX
EXPLAIN ANALYZE SELECT * FROM dwh.fact_observation WHERE tool_key = 57;
-----
QUERY PLAN

Seq Scan on fact_observation  (cost=0.00..308.00 rows=81 width=107) (actual time=0.504..0.889 rows=81 loops=1)
  Filter: (tool_key = 57)
  Rows Removed by Filter: 9919
Planning Time: 0.393 ms
Execution Time: 0.944 ms
(5 rows)

ROLLBACK;
ROLLBACK
EXPLAIN ANALYZE SELECT * FROM dwh.fact_observation WHERE tool_key = 57;
-----
QUERY PLAN

Index Scan using idx_obs_tool on fact_observation  (cost=0.29..10.70 rows=81 width=107) (actual time=0.024..0.033 rows=81 loops=1)
  Index Cond: (tool_key = 57)
Planning Time: 0.193 ms
Execution Time: 0.052 ms
(4 rows)

```

Рисунок 5 - EXPLAIN ANALYZE запроса Q1 до и после создания индекса

Выводы по оптимизации:

- для избирательных запросов (Q1, Q4) индекс заменяет полное чтение таблицы точечным доступом: на 10 тыс. строк ускорение в 24 и 49 раз, на 2 млн - в 2 729 и 23 раз. Время выполнения Q1 с индексом одинаково на обоих объёмах, а время Q4 растёт пропорционально числу найденных строк, а не размеру таблицы;
- частичный индекс для журнала отказов (Q2) и составной индекс (Q3) дают ускорение в 11 и 16 раз на текущем объёме. На 2 млн строк

- выигрыш меньше (2,0 и 2,6 раза): запрос возвращает уже десятки тысяч строк, и основное время уходит на их чтение;
- агрегация по всей таблице (Q5) индексами не ускоряется - СУБД всё равно читает все строки, и планировщик обоснованно выбирает Seq Scan. Для таких запросов при росте объёма следует использовать материализованные представления (агрегаты по классам и циклам), а не индексы;
 - на текущем объёме все запросы выполняются быстрее 3 мс и с индексами, и без них, поэтому основная ценность индексов - запас производительности при росте парка оборудования до требуемых 2 млн записей.

Раздел 6. Выводы

В ходе работы развёрнута СУБД PostgreSQL 16.14 с доступом по паролю, создана база данных ai_project со схемой хранилища из лабы 5, и в неё загружены очищенные данные из лабы 4: 10 000 наблюдений, 382 событий отказа и 120 циклов инструмента. Загрузка выполняется одной программой за 0,8 с в одной транзакции. Факты загружаются командой COPY - самым быстрым из сравнённых способов, в 12 раз быстрее построчной вставки.

Целостность подтверждена на трёх уровнях. Полнота: число записей во всех таблицах совпадает с ожидаемым, всяких ссылок и пропусков в ключах нет. Корректность: статистики и поэлементная сверка всех 10 000 записей совпадают с исходным файлом и результатами лабы 2. Защищённость: шесть видов некорректных вставок отклонены ограничениями СУБД, а повторная загрузка того же файла невозможна.

Перенос в СУБД с точной десятичной арифметикой выявил артефакт исходного набора: 12 меток HDF на границе 8,6 К обусловлены ошибкой округления чисел с плавающей точкой в генераторе данных. Полное совпадение правила HDF с метками в лабе 2 было следствием той же ошибки

в pandas. Для базы знаний сформулирована рекомендация: применять правило в нестрогой форме и снижать коэффициент уверенности для граничных значений.

Созданы семь индексов под типовые запросы системы. Для избирательных запросов они ускоряют выполнение в десятки раз на текущем объёме и в сотни и тысячи раз на нагрузочной копии из 2 млн строк, что обеспечивает выполнение требования масштабируемости из лабы 1. Хранилище готово к использованию базой знаний и модулем оптимизации на следующих этапах проекта.

Список использованных источников

1. Кимбалл Р. Инструментарий хранения и анализа данных. Полное руководство по размерному моделированию / Р. Кимбалл, М. Росс ; пер. с англ. - Москва : Эксмо. - ISBN 978-5-04-108040-2.
2. PostgreSQL 16 Documentation. COPY : [официальная документация]. - URL: <https://www.postgresql.org/docs/16/sql-copy.html> (дата обращения: 05.10.2026).
3. PostgreSQL 16 Documentation. Using EXPLAIN : [официальная документация]. - URL: <https://www.postgresql.org/docs/16/using-explain.html> (дата обращения: 05.10.2026).
4. PostgreSQL 16 Documentation. Partial Indexes : [официальная документация]. - URL: <https://www.postgresql.org/docs/16/indexes-partial.html> (дата обращения: 05.10.2026).
5. Медленно меняющиеся измерения (SCDs) в эпоху облачного хранилища данных // Хабр. - URL: <https://habr.com/ru/companies/slurm/articles/743962/> (дата обращения: 05.10.2026).
6. Что такое DWH (КХД) и как работает корпоративное хранилище данных // Хабр. - URL: <https://habr.com/ru/articles/1037496/> (дата обращения: 05.10.2026).

7. Goldberg D. What Every Computer Scientist Should Know About Floating-Point Arithmetic / D. Goldberg // ACM Computing Surveys. - 1991. - Vol. 23, No. 1. - P. 5-48.

Приложение А. Листинг программы загрузки

"""Загрузка очищенных данных в хранилище PostgreSQL (лабораторная работа 6, вариант 10).

1. Создает схему dwh скриптами из лабы 5 (01_create_schema, 02_reference_data, 03_views).
2. Строит измерения dim_tool, dim_batch и таблицы фактов из data_cleaned.csv (лаба 4).
3. Загружает всё в одной транзакции: измерения - pandas.to_sql, факты - COPY.
4. Сохраняет число загруженных записей в results/load.json.

Параметры подключения берутся из переменных окружения (значения по умолчанию - как в методических указаниях):

DB_HOST=localhost DB_PORT=5432 DB_NAME=ai_project DB_USER=postgres DB_PASSWORD=...

Запуск: python load_dwh.py

"""

```
import hashlib
import io
import json
import os
import time
from pathlib import Path
```

```
import pandas as pd
from sqlalchemy import create_engine, text
```

```
HERE = Path(__file__).resolve().parent
LAB4 = HERE.parent / 'Лаба_4' / 'data'
LAB5 = HERE.parent / 'Лаба_5' / 'sql'
SRC = LAB4 / 'data_cleaned.csv'
RES = HERE / 'results'
RES.mkdir(exist_ok=True)
```

```
DB_NAME = os.getenv('DB_NAME', 'ai_project')
DB_USER = os.getenv('DB_USER', 'postgres')
DB_PASSWORD = os.getenv('DB_PASSWORD', 'mysecretpassword')
DB_HOST = os.getenv('DB_HOST', 'localhost')
DB_PORT = os.getenv('DB_PORT', '5432')
URL = f'postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}'
```

```
CLASS_KEY = {'L': 1, 'M': 2, 'H': 3} # из 02_reference_data.sql
FAILURE_KEY = {'NONE': 0, 'TWF': 1, 'HDF': 2, 'OSF': 3, 'PWF': 4, 'RNF': 5, 'UNKNOWN': 9}
EQUIPMENT_KEY = 1
```

```
def run_sql_file(conn, path):
    """Выполнить SQL-скрипт целиком (несколько команд) через DBAPI-курсор.
    Курсор вызывается без параметров, поэтому символ % в тексте скрипта безопасен."""
    with conn.connection.dbapi_connection.cursor() as cur:
        cur.execute(path.read_text(encoding='utf-8'))
```

```
def copy_df(conn, df, table):
    """Быстрая загрузка DataFrame командой COPY ... FROM STDIN (psycopg2)."""
    buf = io.StringIO()
    df.to_csv(buf, index=False, header=False)
    buf.seek(0)
    cols = ', '.join(df.columns)
    raw = conn.connection.dbapi_connection
    with raw.cursor() as cur:
        cur.copy_expert(f'COPY dwh.{table} ({cols}) FROM STDIN WITH (FORMAT csv)', buf)
```

```
def build_dim_tool(df):
    g = df.groupby('tool_cycle_id')
    tool = pd.DataFrame({
        'tool_key': g.size().index.astype(int),
        'tool_cycle_no': g.size().index.astype(int),
        'first_udi': g['udi'].min().values,
        'last_udi': g['udi'].max().values,
        'products_processed': g.size().values,
        'wear_at_end_min': g['tool_wear_min'].last().values,
        'end_twf': g['twf'].last().values,
    })
    tool['end_reason'] = 'PLANNED'
    tool.loc[tool['end_twf'] == 1, 'end_reason'] = 'TWF'
    tool.loc[tool.index[-1], 'end_reason'] = 'IN_SERVICE' # последний цикл не завершён
    return tool.drop(columns='end_twf')
```

```
def build_fact_observation(df, batch_key):
```

```

f = pd.DataFrame({
    'udi': df['udi'], 'product_id': df['product_id'], 'product_serial':
df['product_serial'],
    'equipment_key': EQUIPMENT_KEY,
    'class_key': df['quality_class'].map(CLASS_KEY),
    'tool_key': df['tool_cycle_id'],
    'failure_type_key': df['failure_mode'].map(FAILURE_KEY),
    'batch_key': batch_key,
})
measures = ['air_temp_k', 'process_temp_k', 'rot_speed_rpm', 'torque_nm', 'tool_wear_min',
            'temp_diff_k', 'power_w', 'power_margin', 'wear_torque', 'osf_margin',
'cycle_pos',
            'machine_failure', 'twf', 'hdf', 'pwf', 'osf', 'rnf_ref', 'n_failure_modes',
            'failure_type_unknown', 'is_outlier_speed', 'is_outlier_torque']
for c in measures:
    f[c] = df[c]
return f.rename(columns={'power_margin': 'power_margin_w'})

def build_fact_failure_event(df, batch_key):
    """Одна строка на каждый отмеченный подтип отказа (включая RNF и UNKNOWN)."""
    parts = []
    for col, code in [('twf', 'TWf'), ('hdf', 'HDF'), ('pwf', 'PWF'), ('osf', 'OSF'),
                     ('rnf_ref', 'RNF'), ('failure_type_unknown', 'UNKNOWN')]:
        sub = df[df[col] == 1]
        parts.append(pd.DataFrame({
            'udi': sub['udi'], 'equipment_key': EQUIPMENT_KEY,
            'class_key': sub['quality_class'].map(CLASS_KEY), 'tool_key':
sub['tool_cycle_id'],
            'failure_type_key': FAILURE_KEY[code], 'batch_key': batch_key,
            'is_primary': (sub['failure_mode'] == code).astype(int),
        })))
    return pd.concat(parts).sort_values(['udi', 'failure_type_key'])

def main():
    t0 = time.perf_counter()
    df = pd.read_csv(SRC)
    md5 = hashlib.md5(SRC.read_bytes()).hexdigest()
    engine = create_engine(URL)
    timings = {}

    # Вся загрузка - одна транзакция: при любой ошибке хранилище остаётся пустым
    with engine.begin() as conn:
        t = time.perf_counter()
        for f in ['01_create_schema.sql', '02_reference_data.sql', '03_views.sql']:
            run_sql_file(conn, LAB5 / f)
            timings['ddl'] = time.perf_counter() - t

        batch_key = conn.execute(text(
            'INSERT INTO dwh.dim_batch (source_name, source_file, source_md5, etl_script) '
            'VALUES (:n, :f, :m, :s) RETURNING batch_key'),
            {'n': 'AI4I 2020 Predictive Maintenance Dataset (cleaned)',
             'f': 'Лаба_4/data/data_cleaned.csv', 'm': md5, 's':
'Лаба_6/load_dwh.py'}).scalar_one()

        t = time.perf_counter()
        dim_tool = build_dim_tool(df)
        dim_tool.to_sql('dim_tool', conn, schema='dwh', if_exists='append', index=False)
        timings['dim_tool'] = time.perf_counter() - t

        t = time.perf_counter()
        fact = build_fact_observation(df, batch_key)
        copy_df(conn, fact, 'fact_observation')
        timings['fact_observation'] = time.perf_counter() - t

        t = time.perf_counter()
        events = build_fact_failure_event(df, batch_key)
        copy_df(conn, events, 'fact_failure_event')
        timings['fact_failure_event'] = time.perf_counter() - t

        conn.execute(text('UPDATE dwh.dim_batch SET rows_loaded = :n WHERE batch_key = :b'),
            {'n': len(fact), 'b': batch_key})
        conn.exec_driver_sql('ANALYZE')

    counts = {}
    with engine.connect() as conn:
        for tbl in ['dim_equipment', 'dim_quality_class', 'dim_tool', 'dim_failure_type',
                    'dim_batch', 'fact_observation', 'fact_failure_event']:
            counts[tbl] = conn.execute(text(f'SELECT count(*) FROM dwh.{tbl}')).scalar_one()

```

```

expected = {'dim_equipment': 1, 'dim_quality_class': 3, 'dim_tool': len(dim_tool),
            'dim_failure_type': 7, 'dim_batch': 1, 'fact_observation': len(df),
            'fact_failure_event': len(events)}
total = time.perf_counter() - t0
print(f'Загрузка завершена за {total:.2f} с')
for tbl, n in counts.items():
    mark = 'OK' if n == expected[tbl] else f'ожидалось {expected[tbl]}'
    print(f' {tbl:20s} {n:>6} {mark}')
json.dump({'counts': counts, 'expected': expected, 'timings': timings, 'total_s': total,
          'source_md5': md5, 'batch_key': batch_key},
          open(RES / 'load.json', 'w'), ensure_ascii=False, indent=1)

if __name__ == '__main__':
    main()

```